

SAP ADT Python - ABAP Development Workbench

Cliente Python para a API REST do SAP ADT com servidor MCP, permitindo que o Cursor IDE funcione como um workbench completo de desenvolvimento ABAP — pesquisar, editar, ativar e gerenciar transports, assim como o Eclipse ADT.

Pré-requisitos

Requisito	Versão mínima	Verificação
Python	3.10+	<code>python --version</code>
pip	23+	<code>pip --version</code>
Git	2.x	<code>git --version</code>
Cursor IDE	—	Instalado com suporte a MCP
Rede	—	Acesso ao host SAP (ex: <code>awsntwpstx01.engdb.infra:8000</code>)

Instalação Passo a Passo

1. Clonar o repositório

```
git clone https://gitlab.engdb.com.br/tax-solution/sap-adt-python.git
cd sap-adt-python
```

2. Criar ambiente virtual Python

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\activate
```

Linux / macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

3. Instalar dependências

```
pip install -r requirements.txt
```

4. Instalar o pacote em modo desenvolvimento

```
pip install -e .
```

Isso registra o pacote `sap_adt` no Python e cria o comando `sap-adt-mcp` no PATH do ambiente virtual.

5. Verificar a instalação

```
python -c "from sap_adt.client import AdtClient; print('OK!')"
```

Configuração do MCP no Cursor

O servidor MCP permite que o Cursor IDE se comunique com o SAP. Há duas formas de configurá-lo:

Opção A — Configuração automática (recomendada)

O repositório já inclui o arquivo `.cursor/mcp.json`. Basta abrir a pasta do projeto no Cursor e o servidor será registrado automaticamente com a chave `sap-adt`.

No painel de MCP do Cursor, ele aparece como `user-sap-adt` (o Cursor adiciona o prefixo `user-`).

Se o servidor MCP não iniciar, substitua `${workspaceFolder}` em `args` pelo caminho absoluto do repositório (ex: `D:\\ENGDB\\sap-adt-python`).

Opção B — Configuração global

Para disponibilizar o MCP em **qualquer workspace** do Cursor, adicione o bloco abaixo no arquivo `~/.cursor/mcp.json`:

Windows: `C:\\Users\\<seu-usuario>\\.cursor\\mcp.json` **Linux/macOS:** `~/.cursor/mcp.json`

```
{
  "mcpServers": {
    "sap-adt": {
      "command": "python",
      "args": [ "D:\\\\ENGDB\\\\sap-adt-python\\\\scripts\\\\run_mcp.py" ],
      "env": {
        "SAP_HOST": "awsntwpstx01.engdb.infra",
        "SAP_PORT": "8000",
        "SAP_CLIENT": "500",
        "SAP_USER": "egoetz",
        "SAP_LANGUAGE": "EN"
      }
    }
  }
}
```

Ajuste o caminho em `args` para o local onde você clonou o repositório. No Linux/macOS use barras normais.

Verificar se o MCP está ativo no Cursor

1. Abra o Cursor IDE
2. Abra a pasta do projeto (`sap-adt-python`)
3. Acesse **Settings** > **MCP** (ou Ctrl+Shift+P → "MCP")
4. O servidor `user-sap-adt` deve aparecer com status verde (ativo)
5. Teste no chat do Cursor: digite `sap_list_environments()` — deve retornar a lista de ambientes salvos

Solução de problemas

Problema	Solução
Servidor não aparece	Verifique se <code>.cursor/mcp.json</code> existe na raiz do projeto
Servidor aparece em vermelho	Verifique o caminho do Python e do <code>run_mcp.py</code> ; veja logs no terminal
<code>ModuleNotFoundError: sap_adt</code>	Execute <code>pip install -e .</code> no ambiente virtual correto
Servidor antigo <code>user-sap-adt-ed2</code>	Remova a entrada antiga de <code>~/cursor/mcp.json</code>
<code>ERROR: Password is required</code>	Execute <code>sap_save_password(...)</code> primeiro (veja seção abaixo)

Configuração Inicial de Credenciais

Antes de conectar ao SAP, salve as credenciais uma única vez. A senha é armazenada no **gerenciador de credenciais do sistema operacional** (Windows Credential Manager / macOS Keychain / Linux Secret Service).

Salvar credenciais (executar no chat do Cursor)

```
sap_save_password(  
    system_id="ED2",  
    host="awsntwpstx01.engdb.infra",  
    port=8000,  
    client="500",  
    user="egoetz",  
    password="SuaSenhaAqui"  
)
```

Conectar ao SAP

```
sap_connect(system_id="ED2")
```

A senha é recuperada automaticamente do keyring do SO — nunca precisa ser passada novamente.

Os parâmetros de conexão ficam em `~/sap-adt/environments.json` (sem senhas). Para remover um ambiente: `sap_delete_password(system_id="ED2")` .

Teste Rápido de Conexão

Para testar a conectividade fora do Cursor:

```
python scripts/test_connection.py
```

Sistemas SAP Configurados

Parâmetro	ED2 (Desenvolvimento)	EQ2 (Qualidade)
Host	awsntwpstx01.engdb.infra	awsntwpstx01.engdb.infra
Porta	8000	8008
Mandante	500	500
Usuário	egoetz	egoetz
Idioma	EN	EN

Ferramentas MCP Disponíveis (21 ferramentas)

Conexão e Credenciais

Ferramenta	Descrição
sap_connect	Autenticar no sistema SAP (suporta ambientes salvos)
sap_discovery	Listar serviços ADT disponíveis
sap_save_password	Armazenar credenciais no gerenciador de credenciais do SO
sap_delete_password	Remover credenciais salvas
sap_list_environments	Listar ambientes SAP salvos

Repositório

Ferramenta	Descrição
sap_search	Pesquisar objetos ABAP (CLAS, PROG, INTF, FUGR, TABL)
sap_browse_package	Listar conteúdo de pacotes (suporta namespaces)
sap_object_metadata	Obter detalhes de um objeto (nome, tipo, pacote, responsável)

Código-Fonte

Ferramenta	Descrição
<code>sap_read_source</code>	Ler código-fonte ABAP (opcional: salvar em arquivo local)
<code>sap_write_source</code>	Gravar fonte com bloqueio automático, transport e ativação
<code>sap_activate</code>	Compilar/ativar objetos
<code>sap_syntax_check</code>	Verificação de sintaxe sem ativar

Gerenciamento de Transports

Ferramenta	Descrição
<code>sap_check_lock</code>	Verificar status de bloqueio do objeto
<code>sap_list_transports</code>	Listar transport requests abertos
<code>sap_create_transport</code>	Criar novo transport request
<code>sap_release_transport</code>	Liberar um transport request
<code>sap_transport_check</code>	Verificar se objeto necessita de transport

Criação e Exclusão de Objetos

Ferramenta	Descrição
<code>sap_create_program</code>	Criar novo programa ABAP
<code>sap_create_class</code>	Criar nova classe ABAP
<code>sap_create_interface</code>	Criar nova interface ABAP
<code>sap_delete_object</code>	Excluir objeto ABAP (requer confirm='DELETE')

Referência completa com parâmetros, exemplos e retornos: [docs/MCP_COMMAND_REFERENCE.md](https://github.com/IBM/ibm-mcp-for-sap/blob/main/docs/MCP_COMMAND_REFERENCE.md)

Arquitetura

Cursor IDE → Servidor MCP (stdio/JSON-RPC) → Cliente REST ADT → SAP /sap/bc/adt/*

O projeto utiliza os mesmos endpoints HTTP que o Eclipse ADT. Todas as operações passam por `/sap/bc/adt/` no sistema SAP.

```
src/sap_adt/
├─ client.py           # Cliente HTTP ADT (autenticação, CSRF, sessões stateful)
├─ credential_store.py # Armazenamento seguro de senhas via keyring do SO
├─ models.py          # Modelos de dados (AbapObject, TransportRequest, etc.)
├─ exceptions.py      # Hierarquia de exceções
├─ repository.py      # Pesquisa e navegação no repositório ABAP
├─ source.py          # Leitura/escrita de código-fonte com gerenciamento de bloqueio
├─ activation.py      # Ativação de objetos, verificação de sintaxe
├─ transport.py       # Gerenciamento de transport requests
└─ mcp_server.py      # Servidor MCP expondo 21 ferramentas
```

Dependências

Pacote	Versão	Finalidade
requests	>=2.31	Cliente HTTP para a API REST ADT
mcp	>=1.0	SDK do Model Context Protocol (FastMCP)
lxml	>=5.0	Parsing de XML para respostas ADT
keyring	>=25.0	Armazenamento seguro de credenciais no keyring do SO

Documentação

Documento	Descrição
docs/PROJECT_DOCUMENTATION.md	Documentação técnica completa (inglês)
docs/PROJECT_DOCUMENTATION_PT-BR.md	Documentação técnica completa (português)
docs/MCP_COMMAND_REFERENCE.md	Referência de comandos MCP com exemplos e pré-requisitos SAP
AGENTS.md	Instruções para agentes Cursor